

ALTSCANNER 자동매매 로드맵

문서 작성일: 2026-05-29 계좌: \$1000 (OKX 서브계정 예정) 기본 비중: $\$100 \times 3x =$ 노출 \$300 / 건당 손실 한도 \$30 동시 오픈 한도: 최대 3건 = 총 노출 \$900 일일 손실 한도: $-\$100$ (계좌의 10%)

📌 핵심 원칙

- 검증 없는 자동매매는 손실을 빠르게 반복하는 기계. 각 단계에서 데이터로 검증 통과한 후에만 다음 단계로 진행.
- 손절은 의지가 아닌 시스템. 거래소 트레일링 스탱으로 강제. 사람이 절대 SL을 풀거나 OFF 못 하게.
- 알트 시장 비대칭성을 무기로. "1건 대박 + 4건 작은 손실" 패턴이 알트의 본질. 분산 + 트레일링으로 활용.
- 단계를 건너뛰지 않는다. 타임라인 단축 < 자금 보존.

📖 전체 단계 요약

Phase	기간	핵심 작업	결과물
0	즉시 ~ 1주	검증 인프라 구축	알림 7일 추적 시스템
1	1일	알림 가이드 강화	진입가/SL/트레일링 표시
2	1~2주	OKX 서브계정 + 수동 진입	30~50건 실거래 데이터
검증 분석	1주	황금비율 찾기	최적 임계값 결정
3	1~2개월	반자동 (Telegram 버튼)	100건 검증 데이터
4	검증 통과 후	완전 자동	무인 운용

총 약 3개월 예상. 무리하게 단축 시 망함.

Phase 0: 검증 인프라 구축

목적: 자동매매가 진짜 돈을 벌지 사전 시뮬레이션으로 검증

작업

1. DB 테이블 — `signal_performance`

sql

```

CREATE TABLE IF NOT EXISTS signal_performance (
  id                bigserial PRIMARY KEY,
  signal_id         bigint REFERENCES signal_log(id),

  symbol            text NOT NULL,
  exchange          text NOT NULL,
  direction         text NOT NULL,
  entry_price       numeric NOT NULL,
  entry_at          timestampz NOT NULL,

  -- 시간 경과별 가격 체크포인트
  price_15m         numeric,
  price_1h          numeric,
  price_4h          numeric,
  price_24h         numeric,
  price_72h         numeric,
  price_7d          numeric,

  -- PnL% (direction 적용 후)
  pnl_15m           numeric,
  pnl_1h            numeric,
  pnl_4h            numeric,
  pnl_24h           numeric,
  pnl_72h           numeric,
  pnl_7d            numeric,

  -- 트레이딩 시뮬레이션
  max_pnl           numeric,
  max_pnl_at        timestampz,
  min_pnl           numeric,
  trailing_exit_price numeric,
  trailing_exit_at   timestampz,
  trailing_pnl       numeric,

  -- SL/TP 히트 여부
  sl_hit            boolean,
  sl_hit_at         timestampz,
  tp_10_hit         boolean,
  tp_20_hit         boolean,
  tp_50_hit         boolean,
  tp_100_hit        boolean,

  created_at        timestampz DEFAULT now(),
  completed_at      timestampz,
  status            text DEFAULT 'tracking',

```

```

    UNIQUE (signal_id)
);

CREATE INDEX idx_perf_symbol_direction ON signal_performance(symbol, direction);
CREATE INDEX idx_perf_entry_at ON signal_performance(entry_at);
CREATE INDEX idx_perf_status ON signal_performance(status);

```

2. `core/tracker.py` — 추적 루프

- 15분마다 status='tracking' 행들 조회
- 거래소 API로 현재가 조회
- 경과 시간에 따라 체크포인트 채움
- max_pnl / min_pnl 업데이트
- SL/TP 히트 여부 체크
- 트레이딩 시뮬: 신고점 -10% 도달 시점 계산
- 7일 경과 시 status='completed' 처리

3. `main.py` 수정

```

python

from core.tracker import tracker_loop

await asyncio.gather(
    # ... 기존 ...
    tracker_loop(), # 알림 후 7일 가격 추적
)

```

결과물

- "현재 임계값 기준 평균 24h 수익률: +X%, 승률 Y%"
- "트레이딩 적용 누적 수익률: +Z%"
- "최적 임계값 조합: cvd ≥ ?, vol ≥ ?"

검증 분석 쿼리

```

sql

```

-- 진단별 성과

```
SELECT
    sl.diagnosis,
    COUNT(*) as n,
    AVG(p.pnl_24h) as avg_pnl_24h,
    AVG(p.trailing_pnl) as avg_trailing_pnl,
    AVG(CASE WHEN p.pnl_24h > 0 THEN 1.0 ELSE 0.0 END) * 100 as win_rate,
    AVG(p.max_pnl) as avg_max_potential
FROM signal_performance p
JOIN signal_log sl ON p.signal_id = sl.id
WHERE p.status = 'completed'
GROUP BY sl.diagnosis
ORDER BY avg_pnl_24h DESC;
```

-- 임계값별 (CVD 구간)

```
SELECT
    CASE
        WHEN ABS(sl.cvd_pct) < 40 THEN '<40'
        WHEN ABS(sl.cvd_pct) < 60 THEN '40-60'
        WHEN ABS(sl.cvd_pct) < 80 THEN '60-80'
        ELSE '80+'
    END as cvd_range,
    COUNT(*) as n,
    AVG(p.pnl_24h) as avg_pnl,
    AVG(CASE WHEN p.pnl_24h > 0 THEN 1.0 ELSE 0.0 END) * 100 as win_rate
FROM signal_performance p
JOIN signal_log sl ON p.signal_id = sl.id
WHERE p.status = 'completed'
GROUP BY cvd_range
ORDER BY cvd_range;
```

-- 트레이일링 vs 24h 고정 청산 비교

```
SELECT
    COUNT(*) as total,
    AVG(pnl_24h) as fixed_24h_pnl,
    AVG(trailing_pnl) as trailing_pnl,
    AVG(max_pnl) as max_potential,
    SUM(CASE WHEN trailing_pnl > pnl_24h THEN 1 ELSE 0 END) as trailing_better_count
FROM signal_performance
WHERE status = 'completed';
```

-- 시간대별 (KST)

```
SELECT
    EXTRACT(HOUR FROM entry_at AT TIME ZONE 'Asia/Seoul') as kst_hour,
    COUNT(*) as n,
    AVG(pnl_24h) as avg_pnl,
```

```
AVG(CASE WHEN pnl_24h > 0 THEN 1.0 ELSE 0.0 END) * 100 as win_rate
FROM signal_performance
WHERE status = 'completed'
GROUP BY kst_hour
ORDER BY kst_hour;
```

Phase 1: 알림 메시지 가이드 강화

목적: 새벽 알림 와도 1분 안에 거래소 세팅 가능

작업

`scorer.py`의 `format_telegram()` 함수에 진입 가이드 섹션 추가.

알림 형식 (예시)

 ALLO-USDT-SWAP - LONG
가격: \$0.1355
CVD: +100% / OI: +51% / Vol: +98%
진단: 숏스퀴즈

 진입 가이드 (\$100 × 3x)
- 진입가: \$0.1355
- 초기 SL: \$0.1220 (-10% / \$30 손실)
- 트레일링 활성화: +10% 도달 시 (\$0.1490)
- 트레일링 폭: -10% (callback rate)

 거래소 입력 순서 (OKX)
1. 시장가 LONG 진입 (\$100 × 3x)
2. Trailing Stop 주문 추가
- Callback rate: 10%
- Activation: \$0.1490
3. 초기 SL: \$0.1220 (Stop Limit)

결과

- 자다 깨도 1분 안에 세팅 끝
- 손절가 직접 계산할 필요 없음

Phase 2: OKX 서브계정 + 수동 진입

목적: 자금 격리 + 수동 진입으로 시스템 친숙해지기

설정 (수동, 코드 X)

1. OKX 서브계정 생성

- 메인 → "서브계정" 메뉴
- 이름: ALTSCANNER-AUTO
- 메인 → 서브로 \$1000 내부 이체

2. 서브 API 키 발급

- 권한 설정:
 - ☒ Trade (필수)
 - ☒ Read (필수)
 - ☒ Withdraw (절대 OFF — 인출 위험 0)
- ☒ IP 화이트리스트: Render outbound IP만 허용
- ☒ Passphrase 설정 (OKX는 API 시크릿 + 패스프레이즈 2단계)

3. 거래 설정

- Isolated Margin (격리 마진) — 한 포지션 청산이 다른 포지션 영향 X
- One-way mode (단방향 모드) — 같은 종목 LONG/SHORT 동시 보유 X

운용 (1~2주)

1. 알림 받으면 서브계정에서 **수동 진입** (Phase 1 가이드대로)
2. 진입 시 입력 항목:
 - 시장가 진입 (\$100 × 3x)
 - Trailing Stop (callback 10%, activation +10%)
 - 초기 SL (-10%)
3. 모든 거래를 매매일지에 기록
4. Phase 0 시뮬 결과와 실제 결과 비교

검증 포인트

- 시뮬 적중률 vs 실제 적중률 일치하는가?
- 거래소 트레일링이 의도대로 작동하는가?
- 수수료/펀딩비 반영 후도 수익인가?
- 30~50건 데이터 모았을 때 평균 PnL 양수인가?

검증 분석 (Phase 2 후)

Phase 3 진입 전 반드시 체크할 것

데이터 분석 항목

1. 임계값 황금비율

- 어떤 cvd/oi/vol 조합이 최고 승률?
- 임계값 조정 → 재테스트

2. 진단별 승률

- "숏스퀴즈" 진짜 강한가?
- "매수 소진" 같은 진단 컷할까?

3. 트레일링 폭 최적화

- -5% / -10% / -15% 중 누적 수익 최대는?
- 활성화 조건 +10%가 최적?

4. 시간대별 차이

- 한국 새벽 vs 미국 장중 적중률 차이?
- 자는 시간 알림은 컷할까?

5. 종합 평가

- 100건 미만이면 표본 부족 → Phase 2 연장
- 평균 PnL 음수 → 시스템 재설계
- 평균 양수 + 승률 35%+ → Phase 3 진입

Phase 3: 반자동 (Telegram 버튼 진입)

목적: 클릭 한 번으로 진입, 사람이 최종 결정

아키텍처

신호 발생

↓

Telegram 알림 (Phase 1 가이드 + 인라인 버튼)

[🚀 진입] [▶ 스킵] [📊 상세]

↓ 사용자 "🚀 진입" 클릭

Telegram Bot Webhook

↓ (Render Background Worker는 inbound HTTP 못 받음)

별도 서비스: Render Web Service (무료 티어 가능)

↓

OKX API:

1. 잔고 확인
2. 동시 오픈 한도 체크 (≤ 3 건)
3. 일일 손실 한도 체크 ($\geq -\$100$)
4. 시장가 진입 ($\$100 \times 3x$)
5. Trailing Stop 주문
6. 초기 SL 주문

↓
trade_journal 자동 기록
↓
Telegram 응답: "✅ ALLO LONG 진입 완료"

작업

1. 새 서비스 1개

- Render Web Service 추가 (Telegram webhook 수신용)
- 또는 Supabase Edge Function

2. OKX API 클라이언트

- Python(ccxt) 라이브러리 또는 직접 구현
- 시장가 진입, Trailing Stop, Stop Limit 주문

3. 안전 장치

```
python

def can_enter(signal):
    if today_loss <= -100: return False
    if open_positions >= 3: return False
    if account_balance < 200: return False
    return True
```

4. Telegram 인라인 버튼 처리

- python-telegram-bot 라이브러리
- 콜백 쿼리 처리

검증 기간

- 코드 작성: 3~5일
- 소액 테스트: 1주 (\$50 × 2x로 시작)
- 안정화: 1주
- 정식 운영: 1~2개월 (100건 데이터)

Phase 4 진입 조건

- ✓ 100건 이상 실제 진입 데이터
- ✓ 평균 PnL: 양수 (수수료/펀딩 차감 후)
- ✓ 승률: 35%+ (대박 1건이 손실 커버 가능 수준)
- ✓ 최대 연속 손실: 10건 이하
- ✓ Kill Switch 실전 검증 완료
- ✓ 본인이 시스템 신뢰

Phase 4: 완전 자동

목적: 사람 개입 없이 신호 발생 시 자동 진입

작업

Phase 3 인프라에서 "사람 버튼 클릭" 제거 + 자동 진입 조건 추가.

python

```
def auto_enter_check(signal):  
    # 안전 장치 (Phase 3와 동일)  
    if today_loss <= -100: return False  
    if consecutive_losses >= 5: return False  
    if open_positions >= 3: return False  
    if account_balance < 200: return False  
  
    # 검증 통과한 화이트리스트만  
    if signal.diagnosis not in DIAGNOSIS_WHITELIST:  
        return False  
  
    # 검증된 임계값 통과  
    if abs(signal.cvd_pct) < OPTIMAL_CVD_THRESHOLD:  
        return False  
    if signal.vol_pct < OPTIMAL_VOL_THRESHOLD:  
        return False  
  
    return True
```

Kill Switch

python

```
# 자동 정지 조건  
- 일일 손실 -$100 도달  
- 연속 5건 손실  
- API 에러 3회 이상  
- 계좌 잔고 $500 미만  
- 수동 OFF (Telegram 명령어 /stop)
```

운용

- 알림은 계속 받되, 진입은 시스템이 결정
- 매일 손익 모니터링 필수
- 예상치 못한 행동 발견 시 즉시 /stop

🏠 인프라 그림 (최종)



⚠️ 위험 요소 & 대응

위험	대응
API 키 유출	IP 화이트리스트 + Withdraw 권한 OFF
시장 급변 (코로나급)	수동 개입 가능 구조 + Kill Switch
검증 데이터 노화	분기마다 황금비율 재검증
수수료/펀딩비	시물예 0.1%/회 + 일 0.3% 차감 반영

위험	대응
코드 버그 (Kill Switch 실패)	Phase 3에서 1주 충분히 테스트
거래소 다운	거래소 네이티브 트레일링 사용 (서버 의존 X)

🚫 절대 하지 말 것

1. 검증 안 한 채 Phase 4 점프
2. 계좌 전체를 서브계정에 넣기 (반드시 격리)
3. API 키 Withdraw 권한 ON
4. 손절 OFF 옵션 만들기 (코드에 절대 X)
5. 레버리지 10x 이상
6. 동시 오픈 5건 이상
7. 검증 결과 무시하고 "감으로" 임계값 결정
8. 단계 건너뛰기

✅ 단계별 체크리스트

Phase 0 (검증 인프라)

- ☐ `signal_performance` 테이블 생성
- ☐ `tracker.py` 작성
- ☐ `main.py`에 `tracker_loop` 추가
- ☐ 배포 후 추적 시작 로그 확인
- ☐ 1주일 후 첫 분석 쿼리 실행

Phase 1 (알림 가이드)

- ☐ `format_telegram` 함수 수정
- ☐ 진입가/SL/트레일링 가이드 추가
- ☐ 실제 알림 텍스트 확인

Phase 2 (서브계정 + 수동)

- ☐ OKX 서브계정 생성
- ☐ \$1000 이체
- ☐ API 키 발급 (Trade+Read, Withdraw X)
- ☐ IP 화이트리스트 설정
- ☐ 격리 마진 + 단방향 모드
- ☐ 수동 진입 30~50건

- ☐ 매매일지에 모두 기록

검증 분석

- ☐ Phase 2 데이터 vs Phase 0 시뮬 비교
- ☐ 임계값 황금비율 결정
- ☐ 진단 화이트리스트 결정
- ☐ 트레이딩 폭 결정
- ☐ 종합 평가 → Phase 3 GO/NO-GO

Phase 3 (반자동)

- ☐ Render Web Service 신설
- ☐ OKX API 클라이언트
- ☐ Telegram 버튼 처리
- ☐ 안전 장치 코드
- ☐ 소액 테스트 1주
- ☐ 100건 데이터 누적

Phase 4 (완전 자동)

- ☐ Phase 3 검증 통과 확인
 - ☐ 자동 진입 조건 정의
 - ☐ Kill Switch 검증
 - ☐ 운영 시작
 - ☐ 매일 모니터링
-

운영 후 정기 점검

매일

- 손익 확인
- 예상치 못한 거래 없는지
- Kill Switch 정상 동작

매주

- 승률 / 평균 PnL
- 트레이딩 동작 여부
- 알림 양 (너무 적/많지 않은지)

매월

- 누적 손익 vs 목표

- 임계값 재검증 (시장 메타 변화)
- 비중 조정 검토

분기

- 황금비율 전면 재검증
 - 인프라 보안 점검 (API 키 갱신)
 - 전략 자체 재평가
-

최종 목표

3개월 후 도달할 상태:

- \$1000 → \$1500~2000 (월 평균 +15~25%)
- 시스템이 새벽 알림 자동 처리
- 손절 일관성 100% (트레일링이 강제)
- 검증 데이터 200건+
- 본인은 모니터링만 (자유 시간 확보)

그 이후:

- 비중 점진적 확대 (\$2000 → \$5000)
 - 알트 외 다른 시장 적용 검토
 - 시스템 완성도가 본인 시간보다 가치 있게
-

이 문서는 살아있는 가이드. 각 단계 진행하면서 발견되는 인사이트는 여기에 계속 업데이트.

마지막 업데이트: 2026-05-29